TABLE OF CONTENT

01 ABSTRACT

02 INTRODUCTION

03 PROBLEM ANALYSIS

04 SOLUTION REQUIREMENTS

05 IMPLEMENTATION OF SOLUTION

06 PROGRAM EXECUTION

07 PROGRAM STRUCTURE FLOW CHART

08 REFLECTION

09 APPENDIX

01 ABSTRACT:

The report details the design, implementation and evaluation of a python-based stroke data Analytics system developed to analyze a simulated data of 172,000 stroke-related patient records. The program is composed of three main modules: dataset _module to load and process data, query_module to query for analysis, ui_module for user interface. And a Jupyter notebook for integration. The report discusses system design decisions, programming process, reflections on challenges, and contributions to professional development.

02 INTRODUCTION:

This report details the development of a Python-based application for analyzing a stroke dataset with 172,000 simulated patient records, designed to assist clinicians in monitoring vital signs and lifestyle factors to prevent strokes. The system is implemented in three modules—dataset_module, query_module, and ui_module—integrated via a Jupyter Notebook. These modules handle data loading, statistical queries, and user interaction, respectively. The report covers problem analysis, solution implementation, program execution, and structure, concluding with a reflection on the development process. The application successfully processes the dataset, computes statistics, and saves outputs as CSV files.

03 PROBLEM ANALYSIS:

The project aims to develop an application that allows clinicians to monitor stroke risk factors and patient outcomes by analyzing a CSV dataset. The dataset includes 21 features such as age, gender, hypertension, heart disease, smoking status, residence type, average glucose level, BMI, sleep hours, dietary habits, and stroke occurrence etc. The system is designed to provide statistical summaries (e.g., mean, median, mode) and case listings based on user queries, helping in clinical decision-making.

The development is divided into three phases:

1. Data Loading Phase: The program reads a CSV file and organizes the data into a nested dictionary, where each record is indexed by a unique ID, and patient attributes are stored as key-value pairs. This structure facilitates efficient data retrieval and querying.
2. Query Processing Phase: The system runs predefined queries to analyze statistics and extract patient cases related to stroke, focusing on factors like hypertension, smoking, heart disease, gender, residence type, and diet. Results are saved as CSV files for further analysis.
3. User interface phase: in this phase we integrate both dataset and query modules to provide a function that implements a friendly and interactive user interface functionality.

The primary challenge is ensuring robust data handling, accurate statistical computations, and an intuitive user interface for query selection and result display. The system must handle missing or invalid data gracefully and provide meaningful outputs for clinical use.

04 SOLUTION REQUIREMENTS:

- The system must fulfill the following requirements:
- Implement a function to load data from a CSV file into a nested dictionary.
- Develop functions to calculate average, median, and modal age for patients with stroke and specific conditions (e.g., hypertension, smoking, heart disease).
- Generate a function to Compute statistics for other numeric features, such as average glucose level and sleep hours.
- Group statistics by categorical variables, such as gender, residence type, and stroke occurrence.
- create a function to Identify dietary habits associated with stroke and non-stroke patients.
- Develop a function to List patient IDs for specific conditions, such as hypertension or heart disease with stroke.
- Create a function to Calculate descriptive statistics (mean, standard deviation, min, max, percentiles) of any of the feature of dataset
- Develop a function to Save query results to external CSV files.
- Function to Provide a user-friendly console interface to select queries and display results.
- Include an exit option to quit the program.

05 IMPLEMENTATION OF SOLUTION:

The stroke data analysis system solution contains four modules—dataset_module.py, query_module.py, ui_module.py, and main. ipynb—designed to load, analyze, and query health data from a CSV file(data.csv), This project focuses on stroke-related

factors such as hypertension, smoking, and heart disease. It features a modular, user-friendly application for statistical analysis and record retrieval via a command-line interface, with results saved as CSV files.

DATASET MODULE:

The dataset_module.py module handles data loading and record lookup. It includes two functions: load_data and test_lookup. The load_data function reads a CSV file (default: data.csv), parsing the first row as headers and subsequent rows as records, storing them in a nested dictionary with incremental IDs (starting from 1) as keys. It uses file I/O with error handling for File Not Found Error, returning an empty dictionary if the file is missing and printing the number of loaded records. The `test_lookup` function is a test function used to provide a command-line interface that prompts users to enter a record ID (or 0 to exit) and displays the corresponding key-value pairs of the record using dictionary access. Example: {'ID': '153575', 'Age': '64', 'Gender': 'Female', 'Hypertension': '1', 'Heart Disease': '0', 'Ever Married': '0', 'Work Type': 'Private', 'Residence Type': 'Rural', 'Average Glucose Level': '298.87', 'BMI': '18.8', 'Smoking Status': 'Formerly smoked', 'Physical Activity': 'Moderate', 'Dietary Habits': 'Vegetarian', 'Alcohol Consumption': '0', 'Chronic Stress': '1', 'Sleep Hours': '11', 'Family History of Stroke': '1', 'Education Level': 'Secondary', 'Income Level': 'High', 'Stroke Risk Score': '26', 'Region': 'East', 'Stroke Occurrence': '1'}

QUERY MODULE:

The query_module.py module contains the core statistical and analytical logic, defining 11 query functions, utility functions, and a batch execution function. Utility functions include save_csv, which writes headers and rows to CSV files using file I/O, and avg, median, and mode, which compute mean, median, and most frequent values using statistical logic. The execute_all_queries function maps query numbers to functions, running them sequentially with exception handling to report success or failure. When run as the main module, it loads the dataset via dataset_module.load_data and executes all queries

USER INTERFACE MODULE:

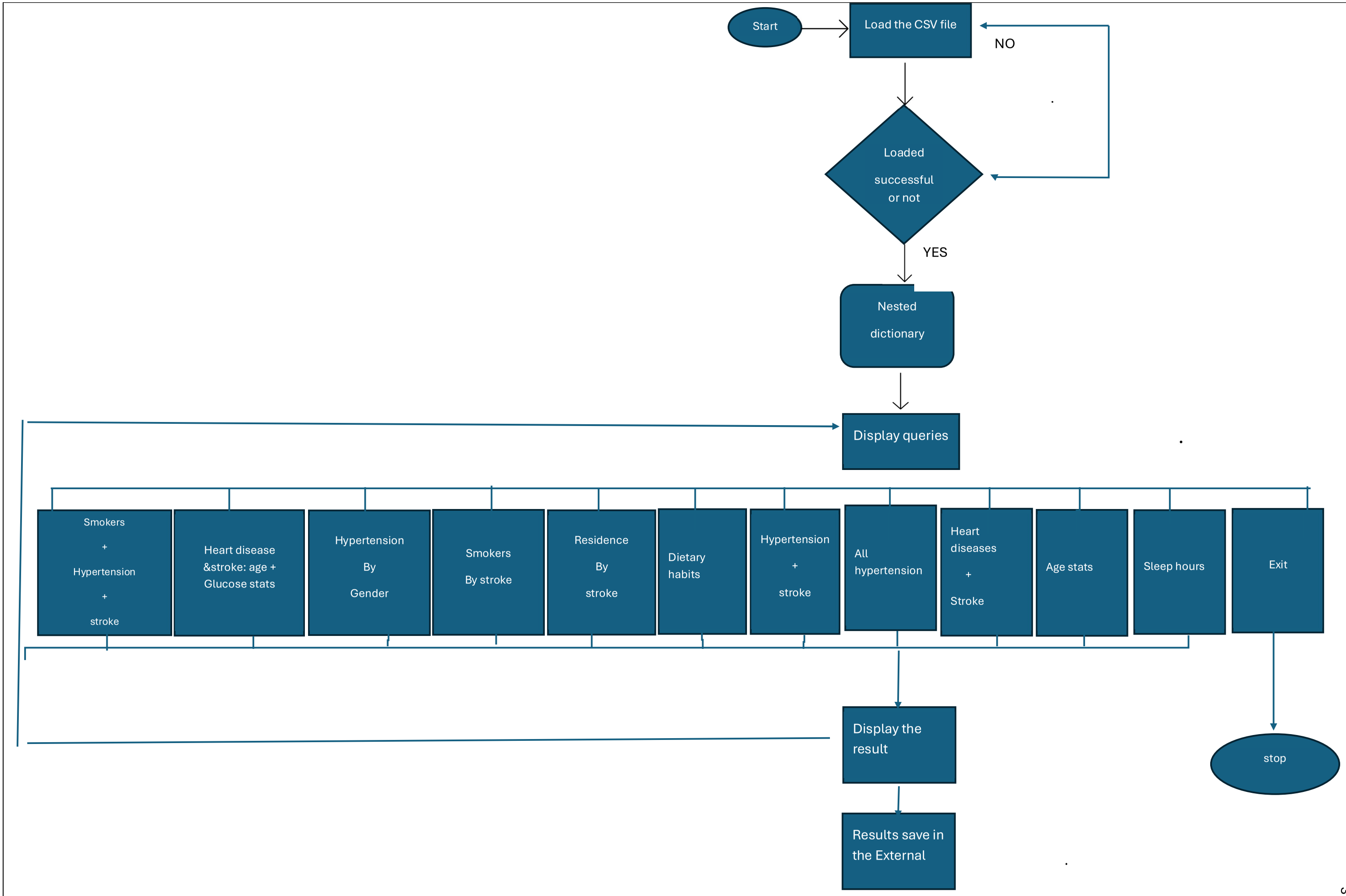
The ui_module.py module provides an interactive command-line interface. It defines four functions: display_query_descriptions, which prints query descriptions from a dictionary; get_query_function, which maps query numbers to query_module.py functions; display_results, which formats outputs (dictionaries, lists, or errors) for readability; and main, which loads the dataset using dataset_module.load_data, displays a menu, and loops to accept user input (query number or 0 to exit). For Q10, it prompts for a feature name. The logic combines user interaction, input validation, and error handling to ensure an spontaneous experience.

The main. ipynb module is a Jupyter Notebook acting as a entry point. It contains one code cell that imports ui_module and calls its main function when executed. Because the Ui module already integration of dataset module and the query module after executing the cell the list of queries are displayed below and when you select a query the result is displayed just below it and the results are saved in external csv file.

06 Program execution:

You can execute the program in both VS CODE and JUPYTER NOTEBOOK. First, make sure that these files: dataset_module, query_module, ui_module, and main. ipynb and data.csv are in same folder (project folder). Before that you have to install jupyter notebook in your system then open the project folder in jupyter notebook and select the main.ipynb file and run the code in the cell then the dataset is loaded and all the queries (1 to 11) will be displayed and you can select any query and then the results will be displayed below the query itself and also the result of that query is saved in external csv file and The queries and the displayed results should be user friendly. The module should allow users to continue with the system as long as they want if you want to exit select 0 to quit when they are done.

07 program structure flow chart:


Start
Load the CSV file
NO
Loaded successful or not
YES
Nested dictionary
Display queries
Smokers + Hypertension + stroke
Heart disease &stroke: age + Glucose stats
Hypertension By Gender
Smokers By stroke
Residence By stroke
Dietary habits
Hypertension + stroke
All hypertension
Heart diseases + Stroke
Age stats
Sleep hours
Exit
Display the result
Results save in the External
stop

08 REFLECTION:

The Development of the stroke data analysis system was satisfying and enlightening, providing a functional system for clinical data analysis while highlighting improvements. The project was robust in its well-defined design with modular structure, segregating data loading, query processing, and user input for maintainability and scalability. Robust error handling ensured stability, and the 11 pre-configured queries returned clinically relevant information, e.g., relationships between hypertension, smoking, and the risk of stroke. Being able to complete deadlines was good planning, leveraging Python's integrated library to create a console system that fulfilled basic objectives. However, problems did exist, including issues with incremental IDs and scalability, query static limitations, and a text interface that can be restrictive for non-technical users. Such issues underscored the need for more flexible, simpler-to-use options in clinical use.

In order to complement the system, a couple of upgrades can be made. Using UUIDs or IDs from the dataset would address scalability, and a dynamic query engine would allow custom analyses based on clinician needs. Shifting towards a web interface through libraries such as Flask would enhance accessibility, with support from data validation stages to offer reliability. Performance optimizations like caching or the use of pandas Data Frames would facilitate efficient large-scale data processing. Advanced features such as alerting important discoveries or predictive modeling would further place the system among cutting-edge healthcare analytics. Such features would bridge technical capability and everyday usability gaps, making the tool immediately accessible for clinical decision-making.

The project yielded abundant learning dividends, pushing Python skill in data manipulation and reinforcing the tenets of modular system design. Clinical data work highlighted the interaction of technical and domain expertise, from statistical computation to stroke risk factor interpretation. Error handling and performance issues like these enhanced problem-solving methods, and console interface limitations led to more efforts to develop more natural, web-based interfaces. These findings are particularly important in the transition to data analytics, where technical strength will have to be balanced against usability. The project also emphasized the importance of training and documentation in facilitating clinicians' use of analysis tools.

In the future, the system is a platform for more advanced healthcare applications. Subsequent versions can include real-time data from electronic health records or utilize machine learning to predict stroke risk, expanding its clinical applications. The lessons—particularly in scalability, usability, and robust data handling—will guide these developments, rendering solutions both innovative and practical. By merging technical acumen with patient outcome focus, such endeavors have the potential to spur significant advancements in digital health. This has only deepened my dedication to using data analytics for healthcare innovation where technology can help translate raw data into meaningful insights for improved care.

APPENDIX:

Pseudocode for dataset_module

```
FUNCTION load_data(filename)
  INITIALIZE empty dictionary data
  IF filename is None
    PROMPT user for filename
  END IF

  TRY
    OPEN file with filename for reading
    READ all lines from file
    IF file is empty
      PRINT "File is empty"
      RETURN empty dictionary
    END IF

    SPLIT first line by comma to get header
    STRIP whitespace from each header element

    FOR each line after header (index i starting from 1)
      SPLIT line by comma to get row
      STRIP whitespace from each row element
      IF row length does not match header length
        PRINT "Skipping row due to inconsistent values"
        CONTINUE
      END IF
      CREATE empty dictionary row_data
      FOR each value in row (index j)
        ADD header[j]: value to row_data
      END FOR
      ADD i: row_data to data
    END FOR
  CATCH FileNotFoundError
    PRINT "File not found"
    RETURN empty dictionary
  CATCH OtherException as e
```

```
      PRINT "An error occurred: e"
      RETURN empty dictionary
    END TRY
    RETURN data
END FUNCTION

IF main program
    CALL load_data with 'data.csv' and store result in data
    PRINT "Successfully loaded 172000 records"
    IF data is not empty
      WHILE true
        TRY
          PROMPT user for row ID (0 to quit)
          IF ID is 0
            BREAK
          END IF
          IF ID exists in data
            PRINT "Data for row ID:"
            FOR each key, value in data[ID]
              PRINT key: value
            END FOR
          ELSE
            PRINT "Row not found"
          END IF
        CATCH ValueError
          PRINT "Invalid input. Enter an integer"
        END TRY
      END WHILE
    ELSE
      PRINT "Failed to load dataset"
    END IF
END IF
```

Pseudocode for query_module

```
FUNCTION load_data(filename)
    TRY
      OPEN file with filename for reading
      READ first line, split by comma, strip and lowercase to get headers
      FOR each subsequent line (index i)
        SPLIT line by comma, strip values
        CREATE dictionary mapping headers to values
```

```
        ADD "row_{i+1}": dictionary to data
      END FOR
      RETURN data
    CATCH Exception
      RETURN None
    END TRY
  END FUNCTION

  FUNCTION filter_data(data, conditions)
    INITIALIZE empty list filtered
    FOR each row in data values
      SET match = true
      FOR each column, value in conditions
        GET cell_value from row[column] (default empty), lowercase and strip
        IF value is a list
          IF no value in list is in cell_value
            SET match = false
          END IF
        ELSE IF value is '1' and cell_value not in ['1', 'yes', 'true']
          SET match = false
        ELSE IF value is '0' and cell_value not in ['0', 'no', 'false']
          SET match = false
        ELSE IF value not in cell_value
          SET match = false
        END IF
      END FOR
      IF match
        APPEND row to filtered
      END IF
    END FOR
    RETURN filtered
  END FUNCTION

  FUNCTION calc_age_stats(data)
    TRY
      COLLECT ages as floats from rows where age exists and is not empty
      IF ages is empty
        RETURN None
      END IF
      COMPUTE average as sum of ages divided by count
      COMPUTE modal as most common age
      SORT ages
```

```
      COMPUTE median based on sorted ages (handle even/odd cases)
      RETURN dictionary with average, modal, median
    CATCH Exception
      RETURN None
    END TRY
  END FUNCTION

  FUNCTION write_to_csv(filename, data)
    TRY
      OPEN file with filename for writing
      IF data is empty
        RETURN true
      END IF
      WRITE headers (data keys) as comma-separated line
      IF data values are lists
        FOR each index in length of first column
          WRITE comma-separated row of values for each header
        END FOR
      ELSE
        WRITE comma-separated row of values for each header
      END IF
      RETURN true
    CATCH Exception
      RETURN false
    END TRY
  END FUNCTION

  FUNCTION query_1(data)
    FILTER data for stroke='1', hypertension='1', smoking in ['1', 'yes', 'true', 'smokes']
    COMPUTE age stats for filtered data
    WRITE stats to 'query1_smokers_hypertension_stroke_age.csv'
    RETURN write success, count of filtered rows
  END FUNCTION

  FUNCTION query_2(data)
    FILTER data for stroke='1', heart disease='1'
    COMPUTE age stats for filtered data
    COMPUTE average glucose level for filtered data
    WRITE stats to 'query2_heart_disease_stroke_stats.csv'
    RETURN write success, count of filtered rows
  END FUNCTION
```

```
FUNCTION query_3(data)
  INITIALIZE empty result dictionary
  FOR gender in ['male', 'female']
    FOR stroke in ['1', '0']
      FILTER data for hypertension='1', gender, stroke
      COMPUTE age stats and add to result with key '{gender}_stroke_{stroke}_{stat}'
    END FOR
  END FOR
  WRITE result to 'query3_hypertension_age_summary.csv'
  RETURN write success, total count of filtered rows
END FUNCTION

FUNCTION query_4(data)
  INITIALIZE empty result dictionary
  FOR stroke in ['1', '0']
    FILTER data for smoking in ['1', 'yes', 'true', 'smokes'], stroke
    COMPUTE age stats and add to result with key 'stroke_{stroke}_{stat}'
  END FOR
  WRITE result to 'query4_smokers_age_summary.csv'
  RETURN write success, total count of filtered rows
END FUNCTION

FUNCTION query_5(data)
  INITIALIZE empty result dictionary
  FOR residence in ['urban', 'rural']
    FILTER data for residence, stroke='1'
    COMPUTE age stats and add to result with key '{residence}_{stat}'
  END FOR
  WRITE result to 'query5_residence_stroke_age_summary.csv'
  RETURN write success, total count of filtered rows
END FUNCTION

FUNCTION query_6(data)
  COLLECT unique dietary habits for stroke='1' and stroke='0'
  CREATE result with conditions ['Stroke', 'No Stroke'] and their habits
  WRITE result to 'query6_dietary_habits_stroke.csv'
  RETURN write success, total count of unique habits
END FUNCTION

FUNCTION query_7(data)
  FILTER data for hypertension='1', stroke='1'
  CREATE result mapping each attribute to list of values from filtered data
```

```
    WRITE result to 'query7_hypertension_stroke_patients.csv'
    RETURN write success, count of filtered rows
END FUNCTION

FUNCTION query_8(data)
    FILTER data for hypertension='1'
    CREATE result mapping each attribute to list of values from filtered data
    WRITE result to 'query8_hypertension_all_stroke_status.csv'
    RETURN write success, count of filtered rows
END FUNCTION

FUNCTION query_9(data)
    FILTER data for heart disease='1', stroke='1'
    CREATE result mapping each attribute to list of values from filtered data
    WRITE result to 'query9_heart_disease_stroke_patients.csv'
    RETURN write success, count of filtered rows
END FUNCTION

FUNCTION query_10(data)
    COLLECT ages as floats from data where age exists
    IF ages is empty
        WRITE empty result to 'query10_age_statistics.csv'
        RETURN write success, 0
    END IF
    COMPUTE mean, std_dev, min, max, 25th, 50th, 75th percentiles
    WRITE stats to 'query10_age_statistics.csv'
    RETURN write success, count of ages
END FUNCTION

FUNCTION query_11(data)
    COLLECT sleep hours for stroke='1' and stroke='0'
    COMPUTE average sleep hours for each group
    CREATE result with conditions ['Stroke', 'No Stroke'] and their averages
    WRITE result to 'query11_sleep_hours_stroke.csv'
    RETURN write success, total count of sleep records
END FUNCTION

FUNCTION query_12(data, results, records)
    CREATE result with query numbers, their status, and records processed
    WRITE result to 'query12_execution_log.csv'
    RETURN write success
END FUNCTION
```

```
FUNCTION execute_all_queries(data)
  INITIALIZE empty results dictionary and records list
  DEFINE list of queries from query_1 to query_11
  FOR each query (index i from 1)
    EXECUTE query and get success, count
    SET results['Query{i}'] to 'Success' or 'Failed' based on success
    APPEND count to records
  END FOR
  APPEND total data length to records
  SET results['Query12'] based on query_12 execution
  PRINT final execution results
  RETURN results
END FUNCTION

IF main program
  CALL load_data with 'data.csv' and store in data
  IF data is None
    EXIT with error
  END IF
  CALL execute_all_queries with data
END IF
```

Pseudocode for ui_module

```
FUNCTION load_data(filename)
  TRY
    OPEN file with filename for reading
    READ first line, split by comma, strip and lowercase to get headers
    FOR each subsequent line (index i)
      SPLIT line by comma, strip values
      CREATE dictionary mapping headers to values
      ADD "row_{i+1}": dictionary to data
    END FOR
    PRINT "Loaded {data length} records"
    RETURN data
  CATCH Exception as e
    PRINT "Error loading file: e"
    RETURN None
  END TRY
END FUNCTION

FUNCTION read_csv(filename, is_multi_row)
```

```
TRY
  OPEN file with filename for reading
  READ all lines
  IF lines count <= 1
    RETURN None
  END IF
  SPLIT first line by comma to get headers
  IF is_multi_row
    FOR each line after header
      SPLIT line by comma to get values
    END FOR
    RETURN dictionary mapping headers to lists of column values
  END IF
  IF only one data line
    RETURN dictionary mapping headers to values
  END IF
  RETURN list of dictionaries mapping headers to values for each line
 CATCH Exception
  RETURN None
 END TRY
END FUNCTION

FUNCTION display_results(results, query_num)
 IF results is empty
  PRINT "No results found"
  RETURN
 END IF
 PRINT separator line
 IF results is dictionary
  FOR each key, value in results
    PRINT key: value
  END FOR
ELSE
  DEFINE fields to display (id, age, gender, etc.)
  COMPUTE max width for each field based on header and data
  PRINT header row with fields
  PRINT separator line
  FOR each row in results
    PRINT row with field values aligned to widths
  END FOR
END IF
PRINT separator line
```

```
END FUNCTION

FUNCTION execute_query_1(data)
  EXECUTE query_1 and check if write succeeded
  IF success
    RETURN result of read_csv('query1_smokers_hypertension_stroke_age.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_2(data)
  EXECUTE query_2 and check if write succeeded
  IF success
    RETURN result of read_csv('query2_heart_disease_stroke_stats.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_3(data)
  EXECUTE query_3 and check if write succeeded
  IF success
    RETURN result of read_csv('query3_hypertension_age_summary.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_4(data)
  EXECUTE query_4 and check if write succeeded
  IF success
    RETURN result of read_csv('query4_smokers_age_summary.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_5(data)
  EXECUTE query_5 and check if write succeeded
  IF success
    RETURN result of read_csv('query5_residence_stroke_age_summary.csv')
  END IF
  RETURN None
END FUNCTION
```

```
FUNCTION execute_query_6(data)
  EXECUTE query_6 and check if write succeeded
  IF success
    RETURN result of read_csv('query6_dietary_habits_stroke.csv', true)
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_7(data)
  EXECUTE query_7 and check if write succeeded
  IF success
    RETURN result of read_csv('query7_hypertension_stroke_patients.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_8(data)
  EXECUTE query_8 and check if write succeeded
  IF success
    RETURN result of read_csv('query8_hypertension_all_stroke_status.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_9(data)
  EXECUTE query_9 and check if write succeeded
  IF success
    RETURN result of read_csv('query9_heart_disease_stroke_patients.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_10(data)
  EXECUTE query_10 and check if write succeeded
  IF success
    RETURN result of read_csv('query10_age_statistics.csv')
  END IF
  RETURN None
END FUNCTION

FUNCTION execute_query_11(data)
  EXECUTE query_11 and check if write succeeded
```

```
  IF success
    RETURN result of read_csv('query11_sleep_hours_stroke.csv', true)
  END IF
  RETURN None
END FUNCTION

FUNCTION show_menu()
  CALL load_data with 'data.csv' and store in data
  IF data is None
    PRINT "Failed to load data. Exiting"
    RETURN
  END IF
  DEFINE queries dictionary mapping numbers to descriptions and execute functions
  WHILE true
    PRINT menu header
    PRINT query options (1-11) and exit (0)
    TRY
      PROMPT user for choice (0-11)
      IF choice is 0
        BREAK
      END IF
      IF choice is valid
        PRINT query header with description
        EXECUTE query function for choice
        IF result is None
          PRINT "Query returned no records"
        END IF
        CALL display_results with result and choice
        WAIT for user to press Enter
      ELSE
        PRINT "Invalid choice"
      END IF
    CATCH ValueError
      PRINT "Enter a valid number"
    END TRY
  END WHILE
END FUNCTION

IF main program
  CALL show_menu
END IF
```